

J A V A W E B P R O G R A M M I N G

Java Web Application Deployment

From JAR to Cloud — packaging, containers, and going live



Aleksander Radovan, Ph.D
Algebra Bernays University

What we'll cover today



Java Archives

JAR · WAR · EAR — what's the difference?

01



Sharing Localhost

Exposing your dev app with Ngrok

03



Docker & Compose

Containerizing your Java app

05



Database Hosting

Free SQL & NoSQL options in the cloud

07



Spring Boot Maven Plugin

Runnable JAR and embedded Tomcat

02



WAR on Tomcat

Building a WAR and deploying it locally

04



Cloud Deployment

Render, Railway, Fly.io, AWS, GCP & more

06



Comparison & Wrap-up

Pick the right tool for the job

08

01

PART ONE

Java Archives

JAR · WAR · EAR — choosing the right package



Why do we package Java applications?

A compiled Java program is a tree of .class files plus resources (HTML, properties, configuration, images). Distributing dozens or hundreds of loose files is fragile and error-prone. Java archives bundle everything — code, dependencies, and metadata — into a single, portable, versioned artifact that can be moved between machines, signed, deployed, or stored in a repository.



One file to rule them all

Code, resources and dependencies in a single ZIP-based file.



Reproducible artifact

Same MD5/SHA on every machine. Easy to version and audit.



Standard target

Build tools, servers and cloud platforms all expect archives.

JAR — Java Archive



What it is

A ZIP-format file that bundles compiled classes, resources, and a META-INF/MANIFEST.MF describing the archive. Default extension: .jar. It's the universal Java distribution format for libraries, command-line tools, and (with Spring Boot) self-contained web applications.

Typical structure

```
myapp.jar
├── META-INF/
│   └── MANIFEST.MF
├── com/
│   └── algebra/
│       ├── Main.class
│       └── Service.class
└── application.properties
```

Use a JAR when...

Library distribution

Shared code published to Maven Central / private Nexus.

Command-line tool

`java -jar tool.jar ...` (single executable artifact).

Modern Spring Boot apps

Self-contained web apps with embedded servlet container.

Microservices

12-factor style apps that run as plain Linux processes.

WAR — Web Application Archive



What it is

A specialized JAR for Java EE / Jakarta EE web applications. Extension: .war. Defined by the Servlet specification. It contains servlets, JSPs, static resources, and a WEB-INF folder with web.xml and the lib/ + classes/ directories. A WAR is deployed into a Servlet container (Tomcat, Jetty, WildFly) that provides the HTTP server and servlet runtime.

Typical structure

```
myapp.war
├── index.html
├── images/
├── WEB-INF/
│   ├── web.xml
│   ├── classes/
│   └── lib/      ← bundled JARs
```

Use a WAR when...

Existing Java EE infrastructure

Company runs Tomcat / WildFly clusters with central ops.

Multi-tenant servlet container

Several apps share one server, JVM, and config.

Strict separation of concerns

Server admins manage runtime; devs ship the app.

Legacy / regulated environments

Banking, telecom, public sector with established platforms.

EAR — Enterprise Application Archive



What it is

A composite archive used by full Jakarta EE application servers (WildFly, GlassFish, WebSphere, WebLogic). Extension: .ear. It groups multiple modules — one or more WARs, EJB JARs, connector RARs, and shared libraries — under a single META-INF/application.xml descriptor. Today rarely seen in greenfield projects; common in legacy banking and ERP systems.

Typical structure

```
myenterprise.ear
├── META-INF/
│   └── application.xml
├── ui.war      ← web module
├── business.jar ← EJB module
└── lib/        ← shared JARs
```

Use an EAR when...

Full Jakarta EE platform

EJB, JMS, JCA, distributed transactions out-of-the-box.

Multiple modules, single deployment

Bundle several WARs and EJB JARs as one unit.

Legacy enterprise systems

Banking, insurance, government ERPs from the 2000s.

Reality check

New projects almost never use EARs in 2026.

JAR vs WAR vs EAR — at a glance

Criterion	JAR	WAR	EAR
Extension	.jar	.war	.ear
Purpose	Library or standalone app	Single web application	Multi-module enterprise app
Needs external server?	No — runs with <code>`java -jar`</code>	Yes — Tomcat, Jetty, etc.	Yes — full Jakarta EE server
Manifest / descriptor	META-INF/MANIFEST.MF	WEB-INF/web.xml (optional)	META-INF/application.xml
Embedded server?	Yes (Spring Boot Tomcat)	No — host provides it	No — host provides it
Typical size	Small to medium	Medium	Large (multiple modules)
Modern relevance	★★★★★ default today	★★★ still common	★ legacy only

Rule of thumb for this course: build JARs by default; use WAR only when the deployment target requires Tomcat.

02

PART TWO

Spring Boot Build & Run

Maven plugin, runnable JAR, embedded Tomcat



spring-boot-maven-plugin

The Spring Boot Maven plugin extends Maven's package phase to produce an executable, self-contained JAR (or WAR) that includes all dependencies and a launcher capable of running the application with `java -jar`. It also exposes goals for running, testing, building OCI container images, and creating layered artifacts for fast Docker builds.

Add it to your pom.xml

```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

<!-- inherited via spring-boot-starter-parent -->
```

What it does

- ✓ **Repackages**
Wraps your code + deps into a runnable fat JAR.
- ✓ **Launches**
spring-boot:run starts the app from sources.
- ✓ **Layers**
Splits the JAR into Docker-friendly layers.
- ✓ **Builds images**
build-image goal uses Cloud Native Buildpacks.

Plugin goals & the build lifecycle

spring-boot:run

Compiles, classpaths, runs your @SpringBootApplication directly.

spring-boot:repackage

Bound to package; transforms the thin JAR into an executable fat JAR.

spring-boot:build-image

Builds an OCI image with Buildpacks — no Dockerfile needed.

spring-boot:start / stop

Starts the app in the background for integration tests.

Common commands



```
# Run from sources
$ mvn spring-boot:run

# Build the fat JAR
$ mvn clean package

# Skip tests for a quick build
$ mvn package -DskipTests

# Build an OCI image (no Docker file)
$ mvn spring-boot:build-image

# Output: target/myapp-1.0.0.jar
```

Runnable JAR + Embedded Tomcat



What 'runnable JAR' means

A Spring Boot runnable (or 'fat') JAR is not a plain JAR. The MANIFEST.MF declares JarLauncher as the Main-Class. Inside, a BOOT-INF/ directory holds your classes and all dependency JARs as nested JARs. A custom classloader can read nested JARs at runtime, so no extraction step is required. The result: one file that 'just runs' on any JRE.



Embedded Tomcat

spring-boot-starter-web brings Apache Tomcat in as a regular JAR dependency. On startup, Spring Boot creates a TomcatServletWebServerFactory, instantiates an embedded Tomcat, configures the connectors and starts it on port 8080 by default — all inside the same JVM as your application code.



Single process — No external server to install or manage.



Pluggable — Swap Tomcat for Jetty or Undertow with one dependency.

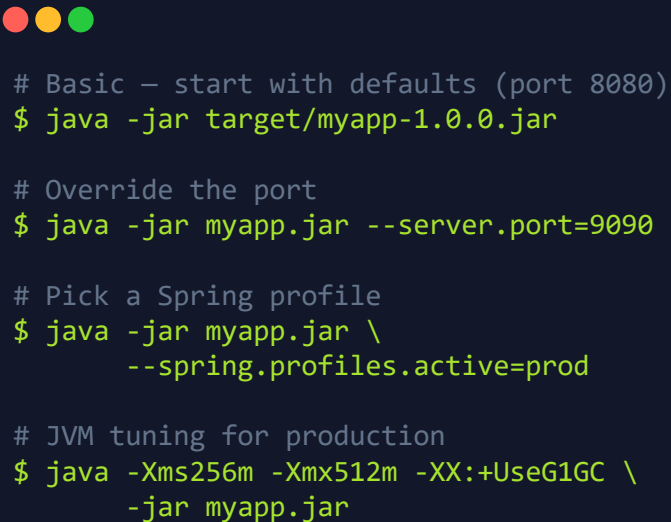


Configurable — server.port, server.servlet.context-path in application.properties.

```
myapp-1.0.0.jar
├── META-INF/MANIFEST.MF
│   └── Main-Class: org.springframework.boot.loader.JarLauncher
├── org/springframework/boot/loader/... ← loader
└── BOOT-INF/
    ├── classes/ ← YOUR code
    └── lib/      ← every dependency JAR
```

Running the application from the JAR

Once mvn package produces target/myapp-1.0.0.jar, you have a portable artifact. Anywhere a recent JRE is installed — your laptop, a colleague's machine, a Linux VM, a container — you can launch it with one command.



```
# Basic — start with defaults (port 8080)
$ java -jar target/myapp-1.0.0.jar

# Override the port
$ java -jar myapp.jar --server.port=9090

# Pick a Spring profile
$ java -jar myapp.jar \
  --spring.profiles.active=prod

# JVM tuning for production
$ java -Xms256m -Xmx512m -XX:+UseG1GC \
  -jar myapp.jar
```

Pro tips

Environment > flags

Read secrets from env vars (SPRING_DATASOURCE_PASSWORD), not the command line.

Background as a Linux service

Wrap the command in a systemd unit; restart on failure.

Health & metrics

Add spring-boot-starter-actuator → /actuator/health endpoint.

Logs to stdout

Let the platform collect them — don't write to local files.

03

PART THREE

Sharing localhost

Exposing your dev server with Ngrok



Ngrok — your localhost on the public internet



The problem

Your Spring Boot app runs at `http://localhost:8080`. A classmate, a client, or a third-party webhook (Stripe, GitHub, Slack) cannot reach localhost — it lives behind your laptop's NAT and Wi-Fi router. You don't have a public IP and you don't want to deploy to a server just for a five-minute demo.



The Ngrok solution

Ngrok is a reverse-tunnel service. A small agent on your machine opens an outbound connection to `ngrok.com` and registers a public URL like `https://abcd1234.ngrok-free.app`. Traffic hitting that URL is forwarded over the tunnel to your local port — no router config, no firewall rules, no DNS changes.

How the tunnel works



Public users / webhook

`https://abcd1234.ngrok-free.app`



Ngrok edge

Authenticates, terminates TLS



Your Spring Boot app

`localhost:8080` on your laptop

Using Ngrok with your Spring Boot app

1

Install the agent

Download from ngrok.com or use a package manager: `brew install ngrok` / `choco install ngrok`.

2

Register and add token

Sign up for a free account, copy your authtoken, run `ngrok config add-authtoken <token>`.

3

Start your Spring Boot app

`mvn spring-boot:run` — confirm it answers at `http://localhost:8080`.

4

Open the tunnel

`ngrok http 8080` — Ngrok prints a public HTTPS URL.

5

Share & test

Hand the URL to a classmate, paste it as a webhook target, or open it on your phone.

What you'll see

```
$ ngrok http 8080

ngrok

Session Status      online
Account             student@algebra.hr
Region              eu (Frankfurt)
Forwarding           https://abcd1234.ngrok-free.app
                    -> http://localhost:8080

Connections  ttl  opn  rt1  rt5
              0   0    0    0
```



Caveats of the free tier

URLs change every restart, a warning interstitial appears, limited concurrent tunnels, public exposure — never tunnel production data.

04

PART FOUR

Traditional WAR deployment

Building a WAR and shipping it to Tomcat



Creating a WAR from a Spring Boot project

By default Spring Boot builds a JAR. To produce a WAR deployable on an external Tomcat, you make three changes: switch the packaging type, mark the embedded Tomcat as provided (the host already has one), and extend `SpringBootServletInitializer` so the servlet container can bootstrap your app.

1. pom.xml

```
<packaging>war</packaging>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-tomcat</artifactId>
  <scope>provided</scope>
</dependency>
```

2. Application bootstrap

```
@SpringBootApplication
public class MyApp
    extends SpringBootServletInitializer {

    public static void main(String[] a) {
        SpringApplication.run(MyApp.class, a);
    }
}
```

3. Build

```
$ mvn clean package
→ target/myapp-1.0.0.war
```

Best of both worlds

The same WAR still runs standalone via `java -jar` — provided scope only hides the Tomcat JARs from the WAR, not from the JVM when launched directly.

Deploying the WAR to local Apache Tomcat

Tomcat layout

```
● ● ●
apache-tomcat-10/
├── bin/
│   ├── startup.sh / startup.bat
│   └── shutdown.sh / shutdown.bat
├── conf/      ← server.xml, tomcat-users.xml
├── logs/      ← catalina.out, access logs
├── webapps/   ← drop your .war here
└── work/     ← compiled JSPs / cache
```

Three ways to deploy

1. Copy & hot deploy

Drop myapp.war into webapps/ — Tomcat auto-deploys.

2. Manager App (UI)

Browse /manager/html, upload the WAR via the web form.

3. Manager API (CLI)

```
curl -u admin:pass -T myapp.war
http://localhost:8080/manager/text/deploy?path=/myapp
```

Quick local deploy

```
● ● ●
$ cp target/myapp-1.0.0.war $CATALINA_HOME/webapps/myapp.war
$ $CATALINA_HOME/bin/startup.sh
$ tail -f $CATALINA_HOME/logs/catalina.out
# Application now reachable at http://localhost:8080/myapp
```

05

PART FIVE

Containerization with Docker

Build once, run anywhere



What is Docker?

Docker is a platform for building, shipping and running applications inside containers. A container is a lightweight, isolated Linux process with its own filesystem, network namespace, and resource limits. The container ships with everything the app needs — JRE, system libraries, application code — so the same image runs identically on a developer laptop, a CI runner, a staging cluster, or a cloud VM.

Key concepts

- ✓ **Image** — A read-only template — layers of files + metadata.
- ✓ **Container** — A running instance of an image — isolated process.
- ✓ **Dockerfile** — Plain-text recipe that defines how to build the image.
- ✓ **Registry** — Storage for images (Docker Hub, GHCR, ECR, GAR).

Container vs VM

	VM	Container
Boot time	Minutes	Seconds
Disk per inst.	GBs	MBs
Isolation	Hardware	Kernel namespaces
Density	Tens	Hundreds per host
Portability	Hypervisor-specific	OCI-standard

Dockerfile for a Spring Boot app

A Dockerfile is a sequence of instructions Docker executes top-down to assemble an image. Each instruction creates a new layer. For Java apps a multi-stage build keeps the final image small: stage 1 compiles, stage 2 ships only the runtime JRE plus the fat JAR.

```
# --- Stage 1: build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q clean package -DskipTests

# --- Stage 2: runtime -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Per instruction

FROM Base image — Maven for build, JRE for run.

WORKDIR Sets working directory inside the container.

COPY Pulls files from your host into the image.

RUN Executes shell command at build time.

EXPOSE Documents which port the app listens on.

ENTRYPOINT Command run when the container starts.

Building and running the Docker image



Build

Reads ./Dockerfile and creates a tagged image cached layer-by-layer.

```
$ docker build -t myapp:1.0 .
```



Inspect

Lists images on the local daemon — REPOSITORY, TAG, IMAGE ID, SIZE.

```
$ docker images
```



Run

Starts a container, maps host port 8080 → container 8080, runs in background.

```
$ docker run -d -p 8080:8080 --name myapp  
myapp:1.0
```



Operate

Follow logs, stop the container, then remove it when finished.

```
$ docker logs -f myapp | $ docker stop  
myapp | $ docker rm myapp
```

Docker Compose — multi-container apps

Real Java web apps rarely run alone. They need a database, a cache, sometimes a message broker. Docker Compose describes a stack of containers in a single docker-compose.yml file: which images, which ports, which networks, which volumes, and which depend on which. One command — docker compose up — starts the whole stack.

```
services:
  app:
    build: .
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/app
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: secret
    depends_on: [db]

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: app
      POSTGRES_PASSWORD: secret
    volumes: ["pgdata:/var/lib/postgresql/data"]

volumes: { pgdata: {} }
```

Useful commands

```
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f app
$ docker compose down -v
```

Why it matters

Reproducible dev env

Same stack on every laptop

Service-aware networking

app talks to db by name

06

PART SIX

Cloud deployment

From Render to AWS, GCP & friends



Render.com — start here



Render

PaaS that 'just works' for Spring Boot — built on AWS, priced like Heroku used to be.

Render is a developer-friendly platform-as-a-service. Connect a Git repository, choose a service type (Web Service, Background Worker, Cron Job, Static Site, PostgreSQL, Redis), and Render builds and deploys on every git push. It speaks both Docker and native buildpacks; for Java you typically supply a Dockerfile.

01

Push code

Spring Boot project + Dockerfile on GitHub or GitLab.

02

New Web Service

render.com → New → Web Service → connect repo.

03

Configure

Region, instance size, env vars (DB URL, secrets).

04

Deploy

Render builds the image and rolls it out — public HTTPS URL.



Free tier

750 monthly instance-hours, free PostgreSQL (90-day expiry on the free DB), free TLS, auto-deploy from Git. The free web service spins down after 15 min of inactivity — fine for student projects, painful for production.

Other free options for Java web apps



Railway

Modern PaaS

Git-based deployments, Nixpacks build, generous starter plan with \$5 monthly credit (≈ 500 hours of a small instance).

- ✓ Smooth UX, built-in PostgreSQL / Redis / MongoDB
- ✗ No truly-free tier — credit-based



Fly.io

Global edge

Runs containers as Firecracker microVMs across 30+ regions. fly launch detects Spring Boot and generates a Dockerfile + fly.toml.

- ✓ Multi-region, persistent volumes, very fast cold-start
- ✗ Free credits removed in 2024 — small pay-as-you-go cost



Koyeb

Serverless containers

Deploy from a Git repo or a Docker image. One free 'Nano' web service + free Postgres. EU-friendly (Frankfurt, Paris).

- ✓ Genuinely free always-on tier, EU regions
- ✗ Small instance (512 MB RAM)



Google Cloud Run

Serverless

Run any container, billed per request. Cold-starts a JVM in ~1 s with AOT or CDS. 2 M requests/month free in perpetuity.

- ✓ Pay only for traffic, scales to zero, fully managed
- ✗ Cold starts for low-traffic apps

Other free options for Java web apps



AWS Elastic Beanstalk

Managed EC2

Upload a Spring Boot JAR or WAR; AWS provisions EC2 + load balancer + RDS hooks. 750h/month free EC2 (12 months).

- ✓ Production-grade, integrates with whole AWS estate
- ✗ 12-month limit, complex billing if you grow



Azure App Service

Managed PaaS

Native support for Java SE, Tomcat, JBoss EAP. Free F1 tier offers 60 CPU-min/day & 1 GB storage. Student license sweeter.

- ✓ Excellent IDE integration, Java is a first-class citizen
- ✗ F1 tier shares CPU — only good for demos



Oracle Cloud (OCI)

Always-Free VM

Two AMD VMs (1 OCPU, 1 GB) + four ARM Ampere VMs (4 OCPU, 24 GB total) — free forever. Run Tomcat or your own JAR.

- ✓ Most generous always-free VM in the industry
- ✗ DIY — you manage the OS, JVM, TLS, firewall



Heroku-style: Choreo, Back4App, Northflank

Heroku heirs

After Heroku killed its free tier, several PaaSes filled the gap. Choreo (WSO2) and Back4App offer free containerized apps + DB.

- ✓ Click-deploy Spring Boot, free DB attached
- ✗ Smaller communities, sometimes fewer regions

Cloud platforms compared

Platform	Always-free?	Deploy model	Cold start	Custom domain + TLS	Best for
Render	Limited	Git push / Docker	Yes (15 min idle)	Free	Student & MVP projects
Railway	Credit-based	Git push / Docker	No	Free	Fast iteration, side projects
Fly.io	Pay-as-you-go	Docker / flyctl	No	Free	Global low-latency apps
Koyeb	Yes — 1 service	Git push / Docker	No	Free	Always-on EU hosting
Google Cloud Run	2M req/month	Container	Yes	Free	Serverless APIs
AWS Beanstalk	12 months	JAR / WAR / Docker	No	Free + Route53	Enterprise, AWS-native
Azure App Service	F1 forever	JAR / WAR / Docker	Yes (F1)	Free (B1+)	Java + .NET ecosystems
Oracle Cloud (OCI)	Yes — generous	VM, self-managed	No	DIY	Heavy workloads on free tier

Heuristic: hobby + demo → Koyeb / Render. Side project that grows → Railway / Fly.io. Serverless API → Cloud Run. Enterprise / regulated → AWS / Azure / OCI.

07

PART SEVEN

Database hosting

Free SQL & NoSQL options in the cloud



Relational (SQL) databases — free tiers



Supabase

Postgres

Hosted PostgreSQL + REST/realtime APIs + Auth. 500 MB DB, 2 free projects.



Postgres + dashboard + APIs in one click



Neon

Postgres

Serverless PostgreSQL with branching (like Git branches for your DB). 0.5 GB free.



Pay-per-use, branches for staging



Aiven

Postgres / MySQL

Managed Postgres, MySQL, Kafka, OpenSearch. 1-month free trial + \$300 credit.



Enterprise reliability, EU regions



PlanetScale

MySQL

Serverless MySQL with Vitess sharding. Hobby tier discontinued in 2024, but cheapest paid tier remains.



Branching, schema diffs, fast at scale



Render Postgres

Postgres

Free 256 MB PostgreSQL bound to your Render account. Auto-expires after 90 days.



Zero-config for Render apps



Clever Cloud

MySQL / Postgres

French PaaS with managed DBs. Free dev plan for small apps. Strong GDPR posture.



EU sovereignty, simple pricing

NoSQL databases — free tiers



MongoDB Atlas

Document

Hosted MongoDB on AWS / GCP / Azure. Free M0 cluster: 512 MB storage, shared CPU. Multiple regions.



Industry-standard document DB



Firestore

Document (Google)

Realtime document store from Google. 1 GB stored + 50k reads/day free in Spark plan.



Best for mobile clients + realtime sync



Redis Cloud

Key-value / cache

Managed Redis by Redis Inc. Free tier: 30 MB DB, ideal for sessions and cache.



Fastest cache in the toolbox



Upstash

Redis & Kafka serverless

Serverless Redis priced per request. 10k commands/day free, perfect for serverless apps.



Pay-per-request, no idle cost



DynamoDB

Wide-column (AWS)

AWS' serverless NoSQL. 25 GB + 25 read/write capacity units free forever.



Tightly integrated with AWS services



Cloudflare KV / D1

KV & SQL at edge

KV: globally replicated key-value store. D1: SQLite at the edge. Both have free tiers.



Runs at the edge, milliseconds worldwide

Putting it all together

Here's a practical mapping from common student / freelance scenarios to the deployment stack that fits best. There is no single right answer — pick the simplest combination that meets your requirements today and is easy to evolve tomorrow.

Scenario	App package	Hosting	Database
Quick demo to classmate	Runnable JAR + Ngrok	Your laptop	H2 in-memory
Student portfolio project	Runnable JAR / Docker	Render or Koyeb free tier	Supabase or Render Postgres
Side project growing traffic	Docker image	Fly.io or Railway	Neon or PlanetScale
Serverless REST API	Docker image	Google Cloud Run	Firestore or Upstash
Enterprise / corporate	WAR + Docker	AWS / Azure / OCI	Managed Postgres or DynamoDB
Legacy modernization	WAR	On-prem Tomcat	Existing corporate DB

✦ *Start simple. Grow only when you must. The 'right' stack is whichever lets you ship today.*

Questions?

JAR · WAR · EAR → Spring Boot Maven plugin → Ngrok → Apache Tomcat → Docker & Compose → Render & friends → Cloud SQL / NoSQL.



Hands-on next session:

Take your existing Spring Boot project. Build a JAR, run it. Build a WAR, deploy to Tomcat. Wrap it in a Dockerfile. Pair it with PostgreSQL via Compose. Push it to Render. Done.